

👉 “From LSTM to Seq2Seq: BiLSTM, Encoder-Decoder & Context Vector (and Why It Fails)”

🧠 1. LSTM & GRU — What Still Goes Wrong?

📌 Core Problems

❌ 1. Sequential Processing

- Step-by-step dependency
- Cannot parallelize

👉 Example:

Code

Sentence: I love learning AI
Process: I → love → learning → AI

≡ fixed length
input & output

❌ 2. Long-Term Dependency Issue (Still exists)

👉 Example:

“The movie that I watched last week was amazing”

Model struggles to connect:

- “movie” → “amazing”

❌ 3. Information Compression

- Hidden state keeps updating
- Old info gets diluted

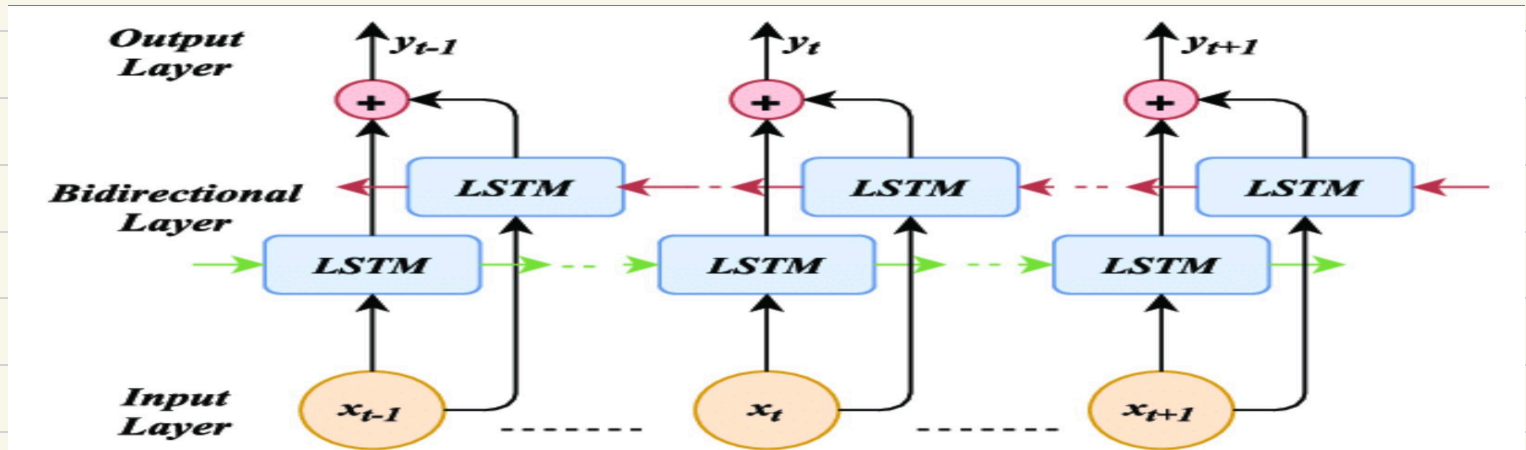
Hidden state
↓
Short term
memory

❌ 4. No Future Context

👉 LSTM reads only left → right



2. BiLSTM – What We Tried to Fix



Idea:

- Process sequence in both directions



Example:

“He saw a bat”

- Forward → “He saw...”
- Backward → “...bat”



Better understanding



Still Problems

- Still sequential
- Still compresses info
- No dynamic focus



3. Why We Needed Sequence-to-Sequence (Seq2Seq)



Problem:

Input length $\neq$ Output length

3. Why We Needed Sequence-to-Sequence (Seq2Seq)

Problem:

Input length $\neq$ Output length

Example:

Translation:

Code

Input: I love AI
Output: J'aime l'IA

✏ Different lengths

✗ LSTM cannot handle this directly

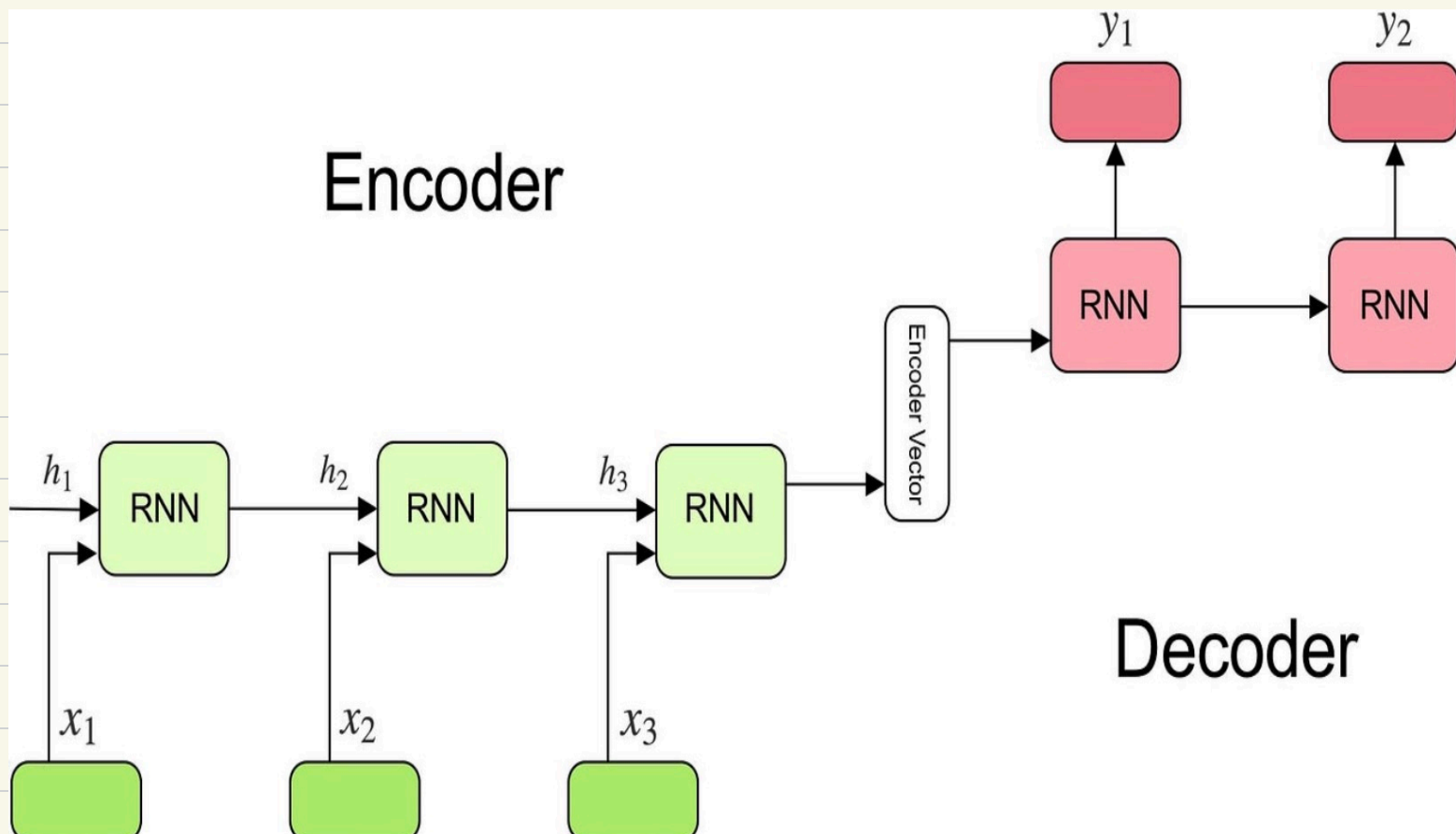
4. Seq2Seq Concept

Idea:

Split into two steps:

1. Understand input
2. Generate output

5. Encoder-Decoder Architecture



◆ Encoder

👉 What it does:

- Reads entire input
- Converts into representation

👉 Example:

Code

I love AI → [vector]

- ① Take input seq. to tokens
- ② Each token is processed using RNN, LSTM, GRU
- ③ final hidden state
(Summarize entire input seq.)
- ④ final state (Context vector)

◆ Decoder

👉 What it does:

- Takes that representation
- Generates output step-by-step

💡 Example:

Code

Vector → J' → aime → l'IA

↓
① Context vector generate the O/P seq. to RNN by token

② Previous O/P to RNN (start)

③ Previous hidden state

④ predicts next word



6. Context Vector (Bridge)



Definition:

A single vector representing entire input



Flow:

Code

Input → Encoder → Context Vector → Decoder → Output



Real Example:

Input:

"The food is amazing"

Context vector stores:

- food
- amazing
- sentiment



7. BIG PROBLEM (Most Important Part 🔥)

Problem 1: Information Bottleneck

- Entire sentence → ONE vector
- Huge compression

→ sender how long input seq.

Problem 2: Long Sentence Failure

👉 Example:

Code

"The food at the restaurant near my office which I visited last week was amazing"

👉 Model forgets early words

→ Does not attend to all word equally, can forget early word in long seq. ☐

Problem 3: No Focus

- Decoder sees same vector always
- Cannot focus on important words

③ how of information

① Difficult in learning alignment b/w input & output (translator prob)

Problem 4: Mixed Information

- Important + unimportant info mixed

🧠💡 Core Realization

👉 Say this slowly:

→ Attention

"We are forcing the model to remember everything in one place... and expecting it to perform perfectly."

🚀 WHY ATTENTION WAS NEEDED

👉 Key Idea Shift:

❌ Old Thinking:

"Compress everything into one vector"

✅ New Thinking:

"Let the model access all information when needed"

⇒ Instead of using the last encoder hidden state as the context vector, the decoder can attend all encoder hidden states & decide which parts to focus on for each old step

Day 12 — Why Attention Replaced RNNs

PART 1: 3 Major Problems with RNNs

Problem 1 — Vanishing Gradient Problem

When RNNs are trained on long sequences, gradients are backpropagated through time (BPTT). At each step, gradients get multiplied by the same weight matrix repeatedly.

Forward Pass:

$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow \dots \rightarrow h_{50} \rightarrow \text{output}$

Backward Pass (gradient flow):

$\text{output} \rightarrow h_{50} \rightarrow h_{49} \rightarrow \dots \rightarrow h_3 \rightarrow h_2 \rightarrow h_1$
 ↓ ↓ ↓
 0.9^2 0.9^3 $0.9^{50} \approx 0.005$

Real Impact:

- Model forgets early tokens in long sequences
- In a 50-word sentence, word 1 barely influences the final output
- LSTMs helped partially but did not fully solve it for very long sequences

Example :-

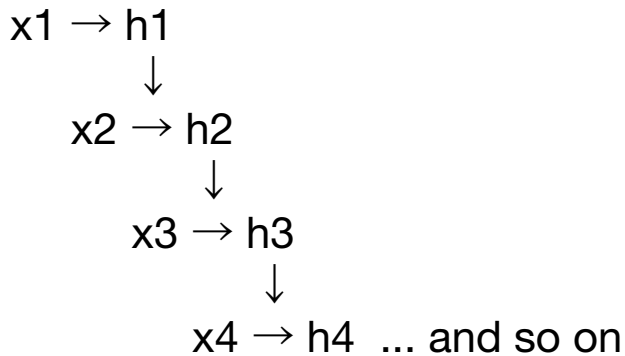
. "The cat that was sitting on the mat near the window in the old house at the end of the street... was hungry."

RNN at "hungry" has almost ZERO memory of "cat"
^

Problem 2 — Sequential Processing Bottleneck

RNNs process tokens one by one. $h(t)$ cannot be computed until $h(t-1)$ is ready.

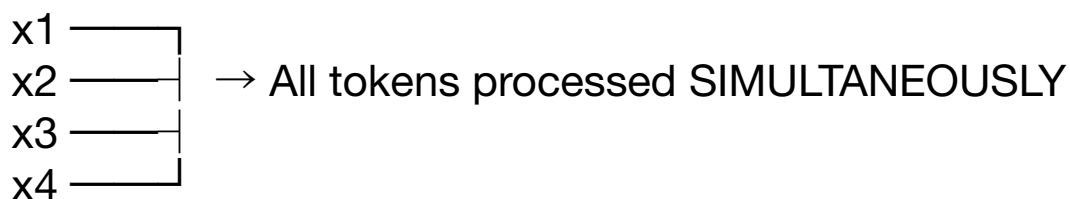
RNN Processing (sequential — cannot parallelize):



Why this is a problem at scale:

- A sequence of 512 tokens = 512 sequential steps
- GPUs are built for parallel computation — thousands of cores sit idle
- Training on millions of sentences becomes extremely slow
- Modern hardware cannot be used efficiently

Transformer (fully parallel):



Real Impact:

GPT-3 would have taken years to train if built on RNNs instead of Transformers.

Problem 3 — Fixed-Size Context Bottleneck

In encoder-decoder RNNs, the entire source sentence is compressed into one single fixed-size context vector.

Encoder:

"I love machine learning"



[h1] → [h2] → [h3] → [h4] → context_vector (e.g. 256 dimensions)



ALL meaning squeezed into one vector

Decoder:

context_vector → "Je" → "aime" → "l'apprentissage" → "automatique"

The Problem:

- A 256-dimensional vector must carry meaning of 50, 100, or 500 words
- Information gets lossy compressed — like fitting a movie into a thumbnail
- The longer the sentence, the worse the output quality

Research Proof:

Bahdanau (2015) showed BLEU scores dropped sharply for sentences longer than 20 to 30 words in RNN-based translation models.

Problem

RNN Behavior

Fix Needed

Vanishing Gradient
Sequential Processing
Fixed Context Vector

Forgets early tokens
Cannot parallelize
Lossy compression

Direct connections
Parallel computation
Dynamic context

☆☆ PART 2: What is Attention Mechanism

What is Attention?

Attention is a mechanism that allows every token in a sequence to look at every other token and decide how much to focus on each one.

Instead of reading a sentence word by word and forgetting early words, attention lets the model focus on the most relevant words — no matter where they appear in the sentence.

Simple Definition:

Attention = Selective Focus + Context Awareness

Human Analogy:

Read this sentence:

"The trophy didn't fit in the suitcase because it was too big."

When your brain reads "it":

- you jump back to "trophy" and "suitcase"
- you compare their sizes
- you decide "it" = trophy

You do NOT re-read every word equally.

You selectively focus on what matters.

That is exactly what attention does — but mathematically.

2. Why Was Attention Needed?

Before attention, RNN encoder-decoder models compressed the entire input sentence into one single fixed-size vector.

"I love machine learning"



[context vector] ← entire meaning squeezed here



"Je aime l'apprentissage automatique"

Problems:

- Long sentences lost information during compression
- Decoder had no way to know which input word to focus on at each step
- Translation quality dropped sharply for sentences longer than 20 words

Attention Solution:

Let the decoder look back at ALL encoder states and dynamically decide what is relevant at each decoding step.

3. Core Idea — Query, Key, Value (Q, K, V)

This is the foundation of attention. Everything is built on this.

YOU enter a library with a QUERY:

"I want books about neural networks"



Every book has a KEY (label on spine):

Book A → "Deep Learning Fundamentals"

Book B → "Python Cooking Recipes"

Book C → "Backpropagation & Gradients"

Book D → "History of Rome"



You COMPARE your query against every key:

Query vs Book A → high match → 0.85

Query vs Book B → no match → 0.02

Query vs Book C → high match → 0.90

Query vs Book D → no match → 0.01



You retrieve VALUE (content) weighted by match score:

Final = $0.85 \times (\text{Book A content})$

+ $0.02 \times (\text{Book B content})$

+ $0.90 \times (\text{Book C content})$

+ $0.01 \times (\text{Book D content})$

→ when embedding pass through attention layer, they get updated using attention scores, becoming contextual (dynamic) embeddings.

Q, K, V Mapping Table

Component	Meaning	Simple Question
Query (Q)	What the current token is looking for	"What do I need?"
Key (K)	What each token advertises about itself	"What do I offer?"
Value (V)	Actual content each token contributes	"Here is my content"

Token Level Example

Sentence: "The cat sat on the mat"

Processing token "sat":

Query of "sat" → asking "who/what is relevant to me?"

Keys from all tokens:

"The" → K1

"cat" → K2 ← high match (subject of sat)

"sat" → K3 ← self

"on" → K4

"the" → K5

"mat" → K6 ← moderate match (location)

Attention Weights after softmax:

"The" → 0.03

"cat" → 0.55 ← highest

"sat" → 0.20

"on" → 0.08

"the" → 0.02

"mat" → 0.12

Output for "sat":

$$\begin{aligned} &= 0.03 \times V_{\text{The}} + 0.55 \times V_{\text{cat}} + 0.20 \times V_{\text{sat}} \\ &+ 0.08 \times V_{\text{on}} + 0.02 \times V_{\text{the}} + 0.12 \times V_{\text{mat}} \end{aligned}$$

= "sat" enriched with context from whole sentence

4. The Math — Step by Step

Step 0 — Input Embeddings

Code



Each **token** is converted to a vector **first**:

```
"cat" → [0.2, 0.8, 0.1, 0.5, ...]    ← 512 dimensions
"sat" → [0.9, 0.1, 0.7, 0.3, ...]
"mat" → [0.3, 0.6, 0.2, 0.8, ...]
```

Similar meaning **words** have similar vectors.
These are learned during training.

Step 1 — Create Q, K, V via Linear Projections

Code



```
Q = X × W_Q
K = X × W_K
V = X × W_V
```

Shapes:

```
X      → (seq_len × d_model)    e.g. (6 × 512)
W_Q    → (d_model × d_k)        e.g. (512 × 64)
W_K    → (d_model × d_k)        e.g. (512 × 64)
W_V    → (d_model × d_v)        e.g. (512 × 64)
```

Result:

```
Q → (6 × 64)
K → (6 × 64)
V → (6 × 64)
```

Why **3** separate matrices?

```
W_Q → learns "what to look for"
W_K → learns "how to present yourself"
W_V → learns "what to actually share"
```

Step 2 — Compute Attention Scores

Code



```
Scores = Q × KT
```

```
Shape: (6 × 64) × (64 × 6) = (6 × 6)
```

Score Matrix:

	The	cat	sat	on	the	mat
The	[2.1	0.3	0.8	0.2	1.1	0.4]
cat	[0.5	3.2	2.8	0.3	0.4	1.2]
sat	[0.4	2.9	2.1	0.7	0.3	1.5]
on	[0.2	0.4	0.6	2.8	0.3	2.1]
the	[1.8	0.3	0.4	0.2	2.9	0.5]
mat	[0.3	1.1	1.4	1.9	0.4	3.1]

```
Cell [i, j] = how much token i wants to attend to token j
```

Step 3 — Scale the Scores

Code



```
Scaled Scores = Scores / √d_k
```

```
Example: d_k = 64 → √64 = 8 → divide all scores by 8
```

Why scale?

Large dot products push softmax into saturation.
Gradients become very small **in** saturation zones.
Dividing by √d_k keeps values **in** a stable range.

Without scaling:

```
Scores: [24, 2, 1]
Softmax: [0.999, 0.0004, ...]
```

With scaling:

```
Scores: [3.0, 0.25, 0.12]
Softmax: [0.65, 0.20, 0.15]
```

↑
much more balanced

Step 4 — Apply Softmax

Code

```
Attention Weights = Softmax(Scaled Scores)
```

```
For "sat" row:
```

```
Raw: [0.4, 2.9, 2.1, 0.7, 0.3, 1.5]
```

```
Scaled: [0.05, 0.36, 0.26, 0.09, 0.04, 0.19]
```

```
Softmax: [0.08, 0.38, 0.28, 0.10, 0.07, 0.17]
```

↑
"cat" highest

↑
"mat" second

Softmax converts raw scores to probabilities that sum to 1

Step 5 — Multiply by Values

Code

```
Output = Attention_Weights × V
```

```
Shape: (6 × 6) × (6 × 64) = (6 × 64)
```

```
For "sat":
```

```
Output_sat = 0.08 × V_The  
+ 0.38 × V_cat ← contributes most  
+ 0.28 × V_sat  
+ 0.10 × V_on  
+ 0.07 × V_the  
+ 0.17 × V_mat
```

= rich contextual vector for "sat"

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \times V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \times V$$

⇒
Attention Score

5. Types of Attention

Type 1 — Self-Attention

Q, K, V all come from the same sequence.

Code

```
Input: "The cat sat on the mat"

Q from: [The, cat, sat, on, the, mat]
K from: [The, cat, sat, on, the, mat] ← same sequence
V from: [The, cat, sat, on, the, mat] ← same sequence

Goal: Every token builds rich understanding of itself
      using all other tokens in the same sentence.

Used in: Encoder of Transformer
```

Type 2 — Cross-Attention

Q comes from one sequence. K and V come from another sequence.

Code

```
Translation task:

Q from: [Je, aime, ...] ← decoder (French so
K from: [I, love, machine, learning] ← encoder output (Eng
V from: [I, love, machine, learning] ← encoder output (Eng

Goal: Decoder asks — "As I generate French,
      which English words should I focus on right now?"

Used in: Decoder of Transformer (encoder-decoder models)
```

Type 3 — Masked Self-Attention (Causal Attention)

Prevents the model from looking at future tokens while generating.

Code

```
Generating: "I love machine learning"

When generating "machine":
  CAN attend to: "I", "love", "machine" ← past + self
  CANNOT attend to: "learning" ← future masked

Mask Matrix (applied before softmax):

      I      love  machine  learning
I      [ 0      -inf    -inf    -inf ]
love   [ 0        0    -inf    -inf ]
machine [ 0        0      0    -inf ]
learning [ 0        0      0      0 ]

-inf → after softmax → 0
(zero attention to future tokens)

Used in: GPT-style decoder-only models
```

6. Multi-Head Attention

Problem with Single-Head Attention

Code

"The bank by the river bank approved my loan"

Single head captures:

"bank" → "loan" (financial meaning) ✓

But misses:

"bank" → "river" (geographical meaning) ✗

Both relationships matter. One head is not enough.

What Multi-Head Attention Does

Run multiple attention heads in parallel.

Each head has its own W_Q , W_K , W_V .

Each head learns a different type of relationship.

Code

Input X

Head 1: W_{Q1} , W_{K1} , W_{V1} → Output1 (syntax)
Head 2: W_{Q2} , W_{K2} , W_{V2} → Output2 (semantics)
Head 3: W_{Q3} , W_{K3} , W_{V3} → Output3 (coreference)
Head 4: W_{Q4} , W_{K4} , W_{V4} → Output4 (position)
... up to h heads

All outputs concatenated:

[Output1 | Output2 | Output3 | Output4 | ...]

↓
Linear Layer (W_O)

↓
Final Output (same shape as input)

Formula

Code

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \times W_O$

where:

$\text{head}_i = \text{Attention}(Q \times W_{Qi}, K \times W_{Ki}, V \times W_{Vi})$

What Each Head Learns

Code

Head Type	What it Attends To
-----------	--------------------

Positional heads	→ nearby tokens (local syntax)
------------------	--------------------------------

Syntactic heads	→ subject-verb relationships
-----------------	------------------------------

Coreference head	→ pronouns to their antecedents
------------------	---------------------------------

Rare word head	→ unusual or important tokens
----------------	-------------------------------

Note: GPT-3 uses 96 attention heads per layer.

7. Attention Visualization Example

Code



Sentence: "The animal didn't cross the street because it wa

Attention weights when model processes "it":

it → The		0.04	
it → animal		0.72	← correctly identifies ref
it → didn't		0.03	
it → cross		0.04	
it → the		0.02	
it → street		0.08	
it → because		0.02	
it → it		0.03	
it → was		0.01	
it → too		0.01	
it → tired		0.01	

Result: Model correctly resolved "it" = "animal"
Learned purely from data. No rules programmed.

8. How Attention Solves the 3 RNN Problems

Code



RNN Problem	How Attention Solves It
Vanishing Gradient	Direct token-to-token connections. Gradient travels in 1 step not 50.
Sequential Bottleneck	All token pairs computed simultaneously via matrix multiplication. Fully parallel.
Fixed Context Vector	No compression. Decoder dynamically attends to all encoder states at each step.

9. Complete Flow — Everything Together

Code



ATTENTION MECHANISM

Input Tokens → Embeddings



Create Q, K, V (via W_Q , W_K , W_V)



Score = $Q \times K^T$



Scale = $\text{Score} / \sqrt{d_k}$



Mask = optional (decoder only)



Weights = $\text{Softmax}(\text{Scale})$



Output = $\text{Weights} \times V$

Full Formula:

$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d_k}) \times V$

